

Features, Capabilities, and Limits — in depth

Written for: you, to know exactly what this system does today, what it deliberately does **not** do, what is **missing**, and — for each gap — **what is stopping it** and what it would take to close. Honest and specific, no hand-waving.

Companion to [DEVELOPER GUIDE.md](#), [SERVER INTERNALS.md](#), [CLIENT INTERNALS.md](#), [SETUP GUIDE.md](#).

1. What works today (shipped and verified)

Server

- **Accounts & auth:** register/login/logout/refresh/"log out everywhere," JWT (header *or* cookie), rotating refresh tokens with replay detection, per-IP/per-user rate limits, account lockout, anti-enumeration (uniform failures + timing), CSRF for cookie clients. Verified by the 31-check `smoke-test.sh`.
- **Ingestion:** single and batch (offline backlog) point upload; out-of-order rejection; one DB transaction per request.
- **Trip detection:** averaged-window movement detection robust to $\pm 30\text{--}100$ m fixes; idle/gap closing; teleport rejection; a background sweeper for abandoned trips.
- **Trip measuring:** Kalman + RTS smoothing $\rightarrow$ distance and top speed within a few percent of truth on test data; noise-trip rejection; `recalculate` for old trips.
- **Admin:** live map of everyone's latest position, online/offline status, per-user trails and trips, trip replay, unlock, recalculate. Role-

gated; trip ids not probeable.

- **Ops:** Docker Compose, health check, structured logs, self-signed TLS via nginx, deployed on AWS EC2 with an Elastic IP.

iOS app

- Sign in / register; **automatic** tracking for a signed-in user.
- Foreground + background + screen-locked tracking.
- Motion-driven GPS accuracy (drive/walk/still) so GPS actually engages on trips.
- Battery saving: low-power positioning when still, 2-min heartbeat, batched uploads, Low Power Mode awareness.
- Durable offline queue (survives dead zones, app kill, session expiry); uploads on reconnect.
- Relaunch after iOS termination / reboot via geofence + significant-change + visits + the `voip` boot mode.
- First-unlock-after-reboot recovery.
- Local notifications: trip summaries, tracking on/off, offline/online, and a "tracking stopped" watchdog.
- Certificate pinning; Keychain-stored session; app icon.

2. Deliberate design limits (working as intended)

These are choices, not bugs. Change them only with intent.

Limit	Why it's set this way	Change by
A trip needs ≥ 100 m and $3\times$ its fix accuracy of travel	Filters out noise and walks to the mailbox	<code>MinTripDistanceMeters</code> , <code>TripExtentAccuracyFactor</code>

Fixes worse than 100 m (server) / 150 m (app) are ignored/dropped	They're noise for trips	MaxAccuracyMeters, maxAcceptedAccuracyMeters
Trip closes after 5 min idle / 15 min silence	Distinguishes a stop from the end of a journey	IdleTimeoutMinutes, GapTimeoutMinutes
Admin marks "offline" after 5 min of server silence	Two missed 2-min heartbeats + slack	RECENT_MS in admin.js
Uploads batched every 30 s (60 s Low Power)	Battery: each upload wakes the radio	uploadInterval, lowPowerUploadInterval
Points older than the newest stored are rejected	An offline replay must not rewrite history	ingestion logic
Single DB instance; migrations at startup	Simplicity for one server	Split for multi-replica (see §4)

3. The hard iOS limits (platform-enforced — the "why it won't do X")

This is the section behind every "can you make it always run" question.

These are enforced by iOS itself; no code, entitlement, background mode, or push overrides them on a normal iPhone.

Want	Status	What is stopping it	Only real fix
Keep running after the user force-quits (swipes away)	Blocked	iOS records "user wants it off" and suppresses relaunch triggers for that app	Supervised/MDM device, or user habit (don't swipe)
Relaunch instantly after reboot while standing still	Best-effort	Only the legacy voip boot-relaunch can do it; modern iOS has weakened it — unverified on your device	Confirm on device; else charger Shortcut; else MDM
A notification that opens the app by itself	Blocked	Notifications can't launch apps; only a user tap can	None on stock iOS
Exact fixed-interval background execution	Blocked	iOS schedules background time at its discretion	Accept jitter, or keep a foreground location session (already done)
VoIP/PushKit to wake a killed app silently	Blocked for this use	Since iOS 13, every VoIP push must report a real CallKit call or iOS kills the app; also needs a paid push cert	Not usable for silent tracking

Standard remote push wakes the app to send location	Blocked when not running	A push to a not-running app only shows a banner; your code doesn't run	Needs the app already backgrounded; can't cover force-quit
--	---------------------------------	--	--

What the app already does to get as close as allowed: foreground/background location session, 150 m geofence relaunch, significant-change + visit relaunch, `voip` boot relaunch attempt, first-unlock recovery, and a watchdog notification to prompt the user when all else fails.

4. Missing features (not built yet) — with the blocker and the path

4.1 Reliability / operations

- **Database backups.** *Missing.* Nothing backs up Postgres; a disk/instance loss loses all history. *Blocker:* none — just not set up. *Path:* nightly EBS snapshot via AWS DLM, or a `pg_dump` cron to S3. **Highest-value next step.**
- **Uptime/error alerting.** *Missing.* No one is told if the server is down. *Path:* CloudWatch alarm on the instance + a health-check monitor hitting `/health`.
- **Automated deploy (CI/CD).** *Missing.* Deploy is a manual `tar | ssh` (which once shipped stray Postman files). *Path:* GitHub Actions building from `main` and deploying only committed code.
- **Multi-instance readiness.** *Partially blocked by design.* Rate limits are in-memory and migrations run at startup, so running two API containers would multiply limits and race migrations. *Path:* a distributed rate limiter (Redis) and a separate migration step.
- **Automated tests in the repo.** *Missing.* Detection/smoothing were validated with throwaway scripts, not committed tests. *Path:* port those into xUnit tests over `TripDetector` / `TrackSmoother` so

regressions are caught.

4.2 Security / accounts

- **Change / reset password.** *Missing* (no endpoint). *Blocker*: none. *Path*: add `POST /api/auth/change-password` (and, if wanted, email-based reset — which needs an email provider).
- **Delete my account / data.** *Missing.* *Path*: an endpoint that cascades user deletion (the DB is already set up for cascade).
- **AWS account hardening.** *Gap*: the account is used via root. *Path*: enable MFA on root, create an IAM admin user for daily work.
- **Real TLS certificate.** *Gap*: self-signed cert on a bare IP, so browsers warn and the app must pin a fingerprint. *Blocker*: needs a domain name. *Path*: point a domain at the Elastic IP and use Let's Encrypt; then the app no longer needs the pin.

4.3 App distribution / longevity

- **App stops after 7 days (free Apple ID).** *Blocker*: Apple's free-signing limit. *Path*: AltStore/SideStore (auto re-sign) or a **paid Apple Developer account** (\$99/yr → 1-year installs, real push, reliable background). This is the single biggest reliability lever for the client.
- **Always-on, tamper-resistant tracking.** *Blocker*: iOS force-quit rule (§3). *Path*: a **supervised/MDM-managed** device — the only supported way to lock an app running and relaunch it after reboot/force-quit. Requires owning/administering the phone and enrolling it (Apple Configurator or an MDM). This is the correct answer if this is a company/fleet phone.

4.4 Product features (nice to have)

- Geofence **arrival/departure alerts** for the admin (server-side, on top of stored points).
- Trip **export** (CSV/GPX) and daily/weekly **summaries**.
- **Data retention** policy (e.g. delete raw points after 90 days, keep

trip rollups) — location history currently grows forever.

- Multiple **admin views**: filter by date, search, per-user stats.
 - **Android client** (the server is platform-neutral; only an iOS client exists).
 - App: real onboarding, per-user privacy controls, a visible trip map in-app.
-

5. Priority order (what to do next)

1. **Database backups** — protect the data before anything else.
 2. **AWS root MFA + an IAM user** — protect the account.
 3. **Change-password endpoint** — the admin password has been shared in chat and can't be rotated in-app yet.
 4. **Decide the distribution model** — paid Apple account (longevity + push) and/or supervised device (true always-on). This decision unblocks most of §3 and §4.3.
 5. **A domain + Let's Encrypt** — removes cert warnings and the pinning maintenance.
 6. **Committed tests + CI/CD** — keep accuracy and deploys safe as the project grows.
-

6. One-paragraph honest summary

The system is a complete, working tracker: a robust server that turns noisy GPS into accurate trips, and an iOS client engineered to keep tracking through backgrounding, offline periods, iOS termination, and reboots — as far as Apple allows. Its real ceilings are **platform rules** (force-quit and silent reboot-relaunch can't be beaten on a normal iPhone) and **distribution** (free sideloading expires weekly). Everything else on the missing list is buildable with no new blockers; backups, account hardening, and a distribution decision are the things worth doing first.